

19. Remove Nth Node From End of List

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: July 10, 2026

Problem Summary

Given the head of a linked list and an integer n , remove the n th node from the end of the list and return the head.

Approach

Calculate the total size of the list, convert the "nth from end" position to "index from start" using $\text{index} = \text{size} - n + 1$, walk to that node while tracking prev, then unlink it. Dummy node handles edge case where head itself is removed.

Algorithm

1. Create dummy, point $\text{dummy.next} = \text{head}$.
2. Calculate size via helper Size() .
3. Compute $\text{index} = \text{size} - n + 1$ (1-based from start).
4. Set $\text{prev} = \text{dummy}$, walk head and prev together until $i == \text{index}$.
5. $\text{prev.next} = \text{head.next}$ — unlink the target node.
6. Return dummy.next .

Key Insights

- $\text{index} = \text{size} - n + 1$ converts from-end position to from-start position — core of this approach.
- prev starts at dummy so that when target is the head node ($\text{index} = 1$), $\text{prev.next} = \text{head.next}$ still correctly removes it.
- Dummy node is essential — without it, removing the head requires a separate special case.
- Two-pass approach: one pass for size, one pass to reach target.

Points to Remember

1. **From-end to from-start conversion:** $\text{index} = \text{size} - n + 1$ — reusable whenever you need to convert a tail-relative position to a head-relative one.
2. **Dummy node for deletion:** always attach a dummy before head when the target node could be the head itself — $\text{dummy.next} = \text{head}$, return dummy.next .
3. **One-pass two-pointer alternative:** advance fast by $n+1$ steps from dummy, then move fast and slow together until $\text{fast} == \text{null}$ — slow is exactly at prev. Cleaner and worth memorizing.
4. $\text{prev.next} = \text{curr.next}$ is the universal node deletion pattern — no need to null out curr.next in Java (GC handles it).

Observations

- Loop condition $i \neq \text{index} \ \&\& \ \text{head.next} \neq \text{null}$ is slightly fragile — $\text{head.next} \neq \text{null}$ is redundant if n is always valid per constraints, but acts as a safety guard.
- One-pass alternative exists: two pointers fast and slow, advance fast by n steps first, then move both until fast reaches end — slow lands at the node before target.
- This two-pass solution is simpler to reason about but the one-pass two-pointer approach is what interviewers typically expect.

Time Complexity & Space Complexity

$O(n)$ — two passes: one for size, one to reach index.

$O(1)$ — only pointers, no extra structure.

Linked List

DescriptionEditorialSolutionsSubmissions

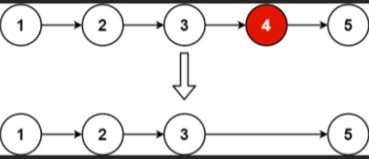
19. Remove Nth Node From End of List

Solved

MediumTopicsCompaniesHint

Given the head of a linked list, remove the n^{th} node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is sz .
- $1 \leq sz \leq 30$

21.4K383188 Online

Accepted

All Submissions

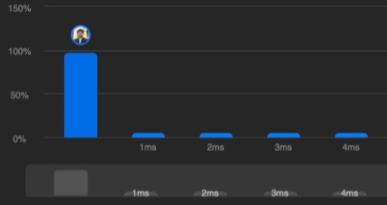
Accepted208 / 208 testcases passed
Sahil Khan submitted at Jul 10, 2026 16:24

Runtime

0 msBeats 100.00%

Memory

43.72 MBBeats 6.75%



Code | Java

```
1 /**
2  * Definition for singly-linked list.
3  * public class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode() {}
7  *     ListNode(int val) { this.val = val; }
8  *     ListNode(int val, ListNode next) { this.val = va
9  *     }
```

View more

More challenges

1721. Swapping Nodes in a Linked List

Code

JavaAuto

```
11 class Solution {
12     public ListNode removeNthFromEnd(ListNode head, int n) {
13         ListNode dummy = new ListNode();
14         dummy.next = head;
15         ListNode prev = dummy;
16
17         int size = Size(head);
18         int index = size-n+1;
19         int i=1;
20
21         while(i!=index && head.next!=null){
22             head = head.next;
23             prev = prev.next;
24             i++;
25         }
26
27         prev.next = head.next;
28
29         return dummy.next;
30     }
31
32     public int Size(ListNode head) {
33         int count = 0;
34         ListNode current = head;
35         while (current != null) {
36             count++;
37             current = current.next;
38         }
39         return count;
40     }
41 }
```

SavedLn 22, Col 26

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3